GUIA TÉCNICO

Fundamentos de Deep Learning

Redes neurais, composição de representações, custo de modelos e avaliação experimental

Propósito — Compreender o que torna um modelo profundo, deduzir custos básicos de arquiteturas densas e diferenciar capacidade expressiva de generalização.

Como utilizar — Leia os fundamentos, acompanhe as contas, execute os exemplos com NumPy e compare as hipóteses antes de realizar experimentos neurais mais custosos.

ROTEIRO DO DOCUMENTO

1. Definições, usos e limites
2. Camadas densas: shapes, parâmetros e custo
3. Não linearidade, XOR e capacidade de representação
4. Experimento executável: três arquiteturas
5. Protocolo de avaliação e armadilhas
6. Exercícios, respostas e referências

01 DEFINIÇÕES, USOS E LIMITES

Deep Learning usa redes parametrizadas que compõem transformações. Para uma entrada x, uma rede de L camadas pode ser representada por f(x) = f_L(...f_2(f_1(x))). Profundidade descreve essa composição; não é sinônimo de desempenho melhor.

Redes densas trabalham com vetores e tabelas; convolucionais exploram estrutura espacial; recorrentes e mecanismos de atenção modelam dependências em sequências. A escolha depende do objetivo, do volume e da natureza dos dados.

Capacidade ≠ generalização

Mais parâmetros ampliam a classe de funções que o modelo pode representar, mas também exigem regularização, dados adequados e avaliação fora da amostra. Ajuste no treino mede adaptação ao conjunto observado; não demonstra utilidade em dados novos.

Quando NÃO começar por uma rede neural?

Se os dados são tabulares, pequenos ou altamente estruturados, métodos lineares e árvores podem fornecer baselines mais simples, rápidos e interpretáveis. Uma rede neural precisa demonstrar ganho sob o mesmo protocolo, e não apenas ter arquitetura mais sofisticada.

Glossário mínimo

Camada — transformação parametrizada (ou operação) aplicada a representações.

Pesos e vieses — parâmetros treináveis ajustados para reduzir a função de perda.

Ativação — função não linear, como ReLU, usada entre transformações.

Época — passagem completa pelo conjunto de treinamento.

Baseline — método de referência a ser superado por comparação justa.

02 CAMADAS DENSAS E CONTAGEM DE PARÂMETROS

Uma camada Dense com n entradas e m unidades calcula z = xW + b, com W de shape (n, m) e b de shape (m,). Uma função de ativação a = g(z) pode ser aplicada em seguida.

Regra de contagem

```
parametros = n_entradas * n_unidades + n_unidades
```

O primeiro termo representa pesos; o segundo, vieses. Para uma sequência de larguras [n0, n1, ..., nL], a soma é Σ (n_i * n_(i+1) + n_(i+1)). Essa soma pressupõe camadas densas com bias e sem compartilhamento de pesos.

Três arquiteturas com oito entradas

Rasa [8,16,1]: (8×16+16) + (16×1+1) = 161 parâmetros.

Profunda [8,8,8,1]: (8×8+8) + (8×8+8) + (8×1+1) = 153 parâmetros.

Larga [8,32,1]: (8×32+32) + (32×1+1) = 321 parâmetros.

Custo, memória e treinamento

Em float32, apenas os pesos treináveis das redes acima ocupam aproximadamente 644, 612 e 1284 bytes. O treinamento requer ainda ativações, gradientes e estados do otimizador: a memória real é maior. Custo de computação cresce com batch, número de exemplos e operações.

Atenção técnica — Dois modelos com contagens próximas podem ter latência e uso de memória distintos. Meça com dados e hardware reais; contagem de parâmetros é uma estimativa parcial de custo.

03 NÃO LINEARIDADE, XOR E REPRESENTAÇÃO

Camadas densas consecutivas sem ativação não linear se reduzem a uma transformação afim equivalente. Portanto, empilhar apenas funções lineares não resolve problemas cuja separação exige fronteiras não lineares.

XOR como contraexemplo

Para pares binários (0,0), (0,1), (1,0) e (1,1), XOR retorna respectivamente 0, 1, 1 e 0. Os pontos positivos ocupam vértices opostos: nenhuma reta pode separar perfeitamente os dois grupos.

```
import numpy as np
X = np.array([[0,0], [0,1], [1,0], [1,1]])
y = np.logical_xor(X[:,0], X[:,1]).astype(int)
assert y.tolist() == [0, 1, 1, 0]
print(X.shape, y.tolist())
```

Representação com duas camadas

Uma camada oculta com ativações não lineares pode formar representações que tornem XOR separável na saída. A existência de uma solução representável não garante que um algoritmo específico de treino a encontrará em cada inicialização.

Experimento controlado sugerido

Compare regressão logística com uma MLP de pequena largura usando as mesmas entradas e uma divisão de avaliação justificada. Para quatro pontos, amostras de teste são insuficientes para estimar desempenho geral: trate o resultado como demonstração matemática.

04 EXPERIMENTO: ESTRUTURAS E SHAPES

O código abaixo reproduz a comparação de arquiteturas do notebook 01 usando apenas Python. Ele verifica contagem de parâmetros, sem treinar redes neurais nem alegar superioridade preditiva.

```
def parametros_densas(larguras):
    if len(larguras) < 2 or any(v <= 0 for v in larguras):
        raise ValueError("Larguras devem ser positivas")
    return sum(a*b+b for a,b in zip(larguras[:-1], larguras[1:]))

modelos = {"raso":[8,16,1], "profundo":[8,8,8,1],
           "largo":[8,32,1]}
for nome, camadas in modelos.items():
    print(nome, parametros_densas(camadas))
```

Resultados esperados e interpretação

Raso: 161; profundo: 153; largo: 321. O modelo profundo do exemplo tem menos parâmetros que o raso, mostrando que profundidade e tamanho não são equivalentes. Nenhuma das três configurações foi treinada: não há acurácia a comparar.

Teste de consistência

```
assert parametros_densas([8,16,1]) == 161
assert parametros_densas([8,8,8,1]) == 153
assert parametros_densas([8,32,1]) == 321
```

Limitações do cálculo

Bias desativado, normalização, embeddings, convoluções e compartilhamento de pesos alteram a contagem. Consulte o sumário real da arquitetura e valide shapes antes de assumir que uma fórmula de Dense descreve a rede inteira.

05 PROTOCOLO DE AVALIAÇÃO

1. Defina o objetivo, os dados e uma métrica adequada antes do treino. 2. Estabeleça um baseline simples. 3. Separe treino, validação e teste sem vazamento. 4. Ajuste transformações somente no treino. 5. Selecione hiperparâmetros na validação. 6. Use o teste apenas para o relatório final.

Exemplos em XOR demonstram expressividade, mas não fornecem estimativa confiável de generalização. Para comparar arquiteturas, fixe orçamento computacional, partição, pré-processamento e condições de treino; documente as sementes e a variabilidade observada.

06 EXERCÍCIOS E SOLUÇÕES COMENTADAS

1. Quantos parâmetros tem uma Dense 12 → 5?

12 × 5 + 5 = 65 parâmetros (com bias).

2. Por que camadas lineares empilhadas não resolvem XOR?

A composição de transformações afins permanece afim; é necessária não linearidade.

3. O modelo [8,8,8,1] é melhor que [8,16,1]?

Não se pode concluir sem treinamento e avaliação comparável; menor contagem não mede desempenho.

Referências para atualização

Keras Guides (keras.io/guides); TensorFlow Guide (tensorflow.org/guide); scikit-learn User Guide (scikit-learn.org/stable/user_guide.html). Consulte as versões efetivas ao executar laboratórios neurais.

Escopo de validação — Este guia contém exemplos verificáveis sem TensorFlow. Treinamentos, resultados e desempenho de redes reais permanecem sujeitos aos testes locais no JupyterLab.

REVISÃO DIDÁTICA · CAPÍTULO 01

Profundidade, generalização e custo

1. Conceito, escopo e interpretação

Profundidade significa compor transformações com parâmetros e ativações intermediárias. Mais camadas aumentam a expressividade possível, mas também elevam custo, sensibilidade à inicialização e necessidade de regularização. Em um experimento comparativo, mantenha iguais os dados de treino/teste, a métrica e o orçamento aproximado de otimização.

2. Formalização e exemplo resolvido

> Dense(d,h): pesos = d*h; vieses = h; total = (d+1)*h. Exemplo: 10 entradas -> 16 neurônios -> 3 saídas: (10+1)*16 + (16+1)*3 = 227 parâmetros.

3. Relação com o notebook e experimento

No estudo dirigido, estime parâmetros de duas arquiteturas e justifique se a variação na complexidade é proporcional à eventual melhora de validação. Documente a semente, a métrica do conjunto de teste e os limites da amostra.

4. Limitações e erros frequentes

Perda menor no conjunto de treino não prova generalização. Diferencie capacidade representacional, desempenho observado e custo computacional.

5. Exercício comentado

Uma Dense de 20 entradas com 10 unidades possui 210 parâmetros; adicionar 2 unidades na saída acrescenta (10+1)*2=22. Quantos parâmetros existem no total? 232.

6. Ampliação conceitual do guia

Uma rede profunda compõe transformações não lineares; profundidade não garante desempenho melhor. Para uma camada densa com d entradas e h unidades, conte d*h pesos e h vieses: (d+1)*h parâmetros treináveis. O crescimento de parâmetros afeta memória, tempo e risco de sobreajuste.

7. Orientação de estudo e validação

No estudo dirigido do notebook, compare arquiteturas com sementes e divisões de dados equivalentes. Relate complexidade, métricas no conjunto de teste e custo computacional; não conclua superioridade só pela perda de treino.

8. Exercício adicional de fixação

Exercício: calcule os parâmetros de duas camadas Dense, uma 10→16 e outra 16→3. Resposta: 176 + 51 = 227 parâmetros.

Verificação: execute o notebook homônimo no ambiente apropriado; registre versões, shapes, sementes e métricas. Laboratórios Keras opcionais não foram executados durante esta revisão documental.

Abrir notebook do capítulo no repositório